Contents

1 - Upgradeability 2
2 - Registry launch (settings protocol) 2
3 - Registry change — Propose (settings protocol) 3
4 - Registry change — Apply (upgrade, settings protocol) 4
5 - Forced spend (beacon indirection) 5
6 - Opt-in native action (Tick) 6
7 - Opt-in upgrade (migration) 7

1 - Upgradeability

Cardano validators are immutable and content-addressed: a validator's address is its code hash, so its logic can never change in place. This protocol provides the two eUTxO-native ways to evolve a deployed protocol, both built on the **Config / Parameter Management** primitive:

- **Migration** (opt-in) — deploy new immutable code and let each user move **their own** UTxO to it.
- **Beacon indirection** (forced) — a thin forwarding validator delegates its logic to a script hash held in a governed **registry**, so the logic can be repointed without moving funds.

Both share a single registry `R`, stored as the settings UTxO's `current` value:

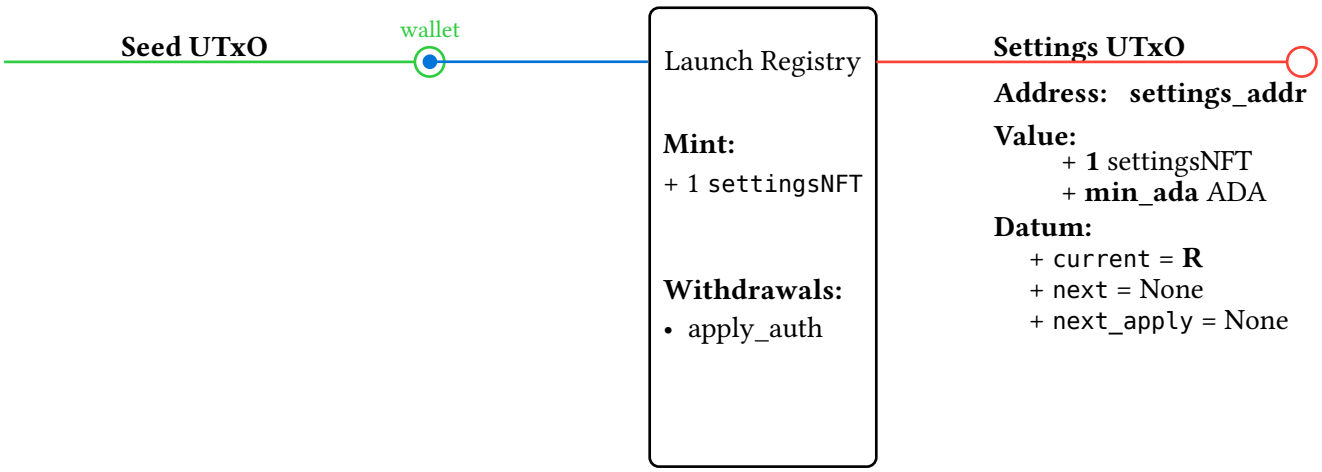
$$R = \{h_{\text{proto}} : h_{\text{delegate}}, \ldots\}$$

where each key `h_proto` is the immutable hash of a protocol validator, and its value `h_delegate` is the script that currently governs it. Registry changes reuse the settings protocol's `Propose` / `Apply` split, its `apply_delay` timelock, and its pluggable `propose_auth` / `apply_auth` authorization.

The registry is a settings value: launching it, proposing a change, and applying a change are the **settings** protocol's own transactions (20), reproduced here for completeness — see `../settings/design.typ` for the full lifecycle. Only the value is registry-shaped. The mechanism-specific transactions follow.

2 - Registry launch (settings protocol)

Creates the settings UTxO that holds the registry. One instance governs any number of protocols.

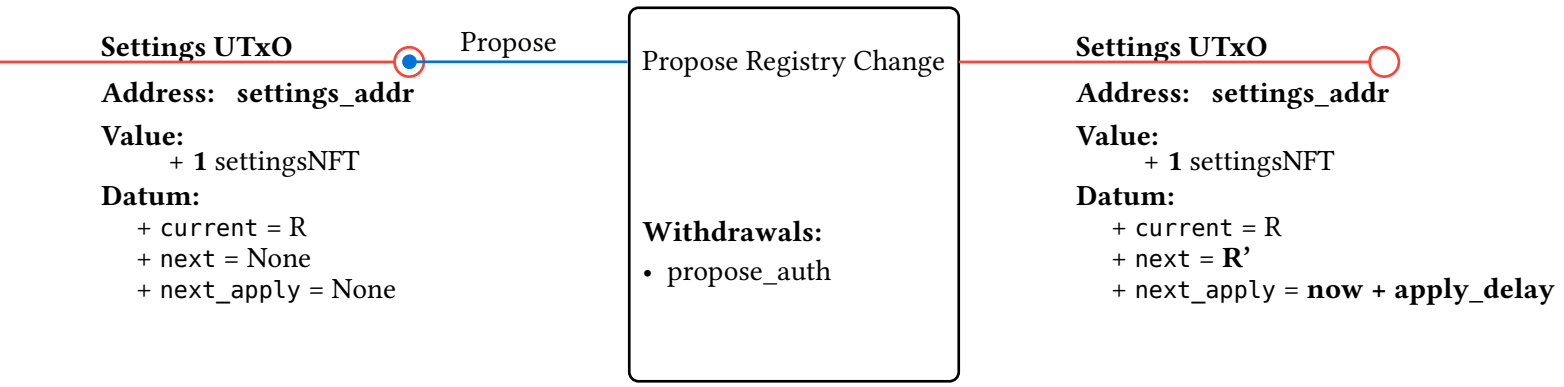


Notes:
- `R` is the registry: a map from a protocol's own hash to its active delegate script hash.
- `settingsNFT` uniquely identifies the instance; minting requires spending the one-shot `seed_utxo`, so at most one instance exists.
- The reference settings validator accepts any value; consumers reject a malformed registry at spend time (`decode_registry`).

Figure 1: Launch registry instance

3 - Registry change — Propose (settings protocol)

Stages a new registry behind the apply delay. A new proposal supersedes any pending one.

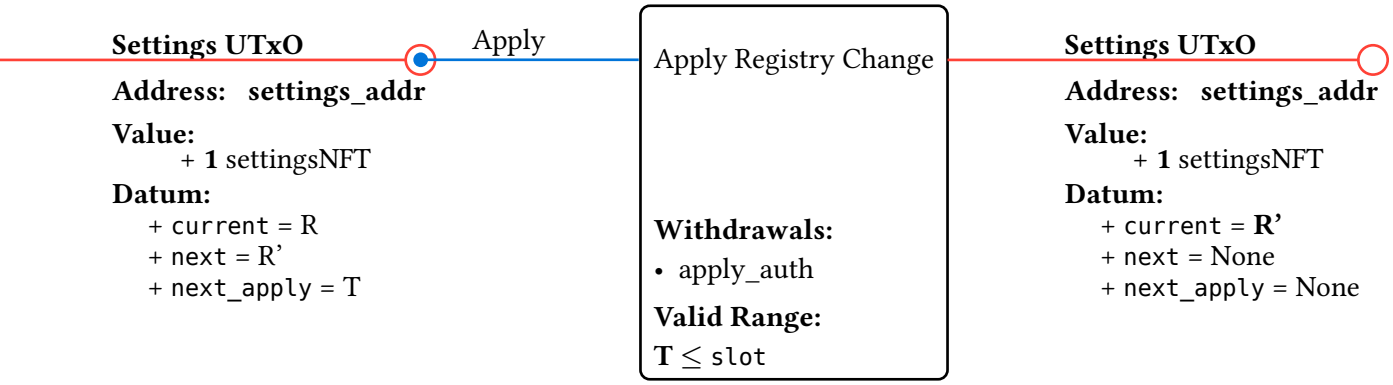


Notes:
- `R' != R` and `validate_datum(R')`.
- The proposal can only change the registry; it cannot be applied yet.

Figure 2: Propose registry change

4 - Registry change — Apply (upgrade, settings protocol)

Commits a matured proposal after the timelock. This is the only transaction that changes a protocol's behaviour.



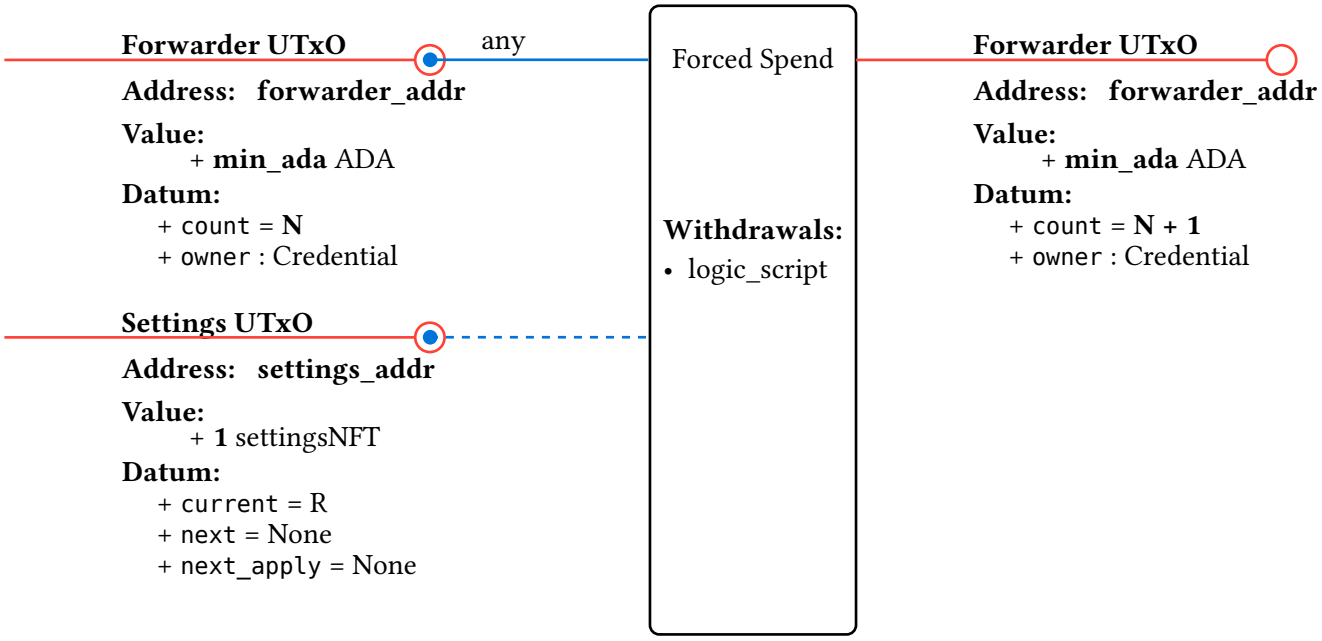
Notes:

- Repointing `R[h_proto] = h_new` **is** the upgrade. For **forced**, `h_new` is the new logic script — funds never move and the change applies to everyone at once. For **opt-in**, `h_new` is the new migration script — each user then chooses to migrate.
- Timelocked: valid only when `now >= T = propose_time + apply_delay`, giving users a window to inspect the new delegate (and, for opt-in, to migrate).

Figure 3: Apply registry change

5 - Forced spend (beacon indirection)

A thin forwarder holds state but carries no logic. Every spend must be accompanied by the logic script registered for the forwarder's own hash.



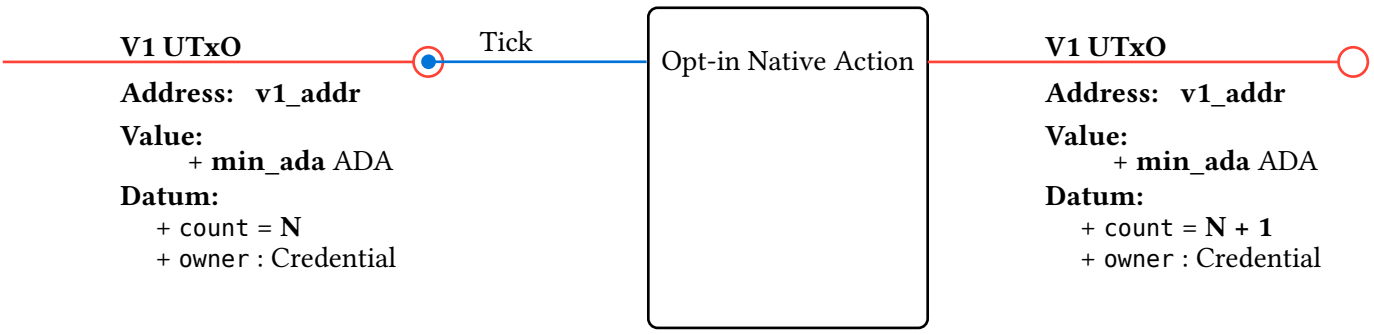
Notes:

- `validate_forward`: `own_hash = forwarder_addr`, `R[own_hash] = logic_script`, and `logic_script` must be present as a withdrawal (withdraw-0 convention).
- The forwarder's redeemer is opaque: the **logic script** is responsible for validating **every** input forwarded to it, not just one.
- A registry repoint changes behaviour for all users at once — this is a **forced** upgrade, so timelock and bounded governance are mandatory.

Figure 4: Forced spend transaction

6 - Opt-in native action (Tick)

An opt-in protocol keeps its native logic for ordinary actions; only the `Upgrade` path forwards.

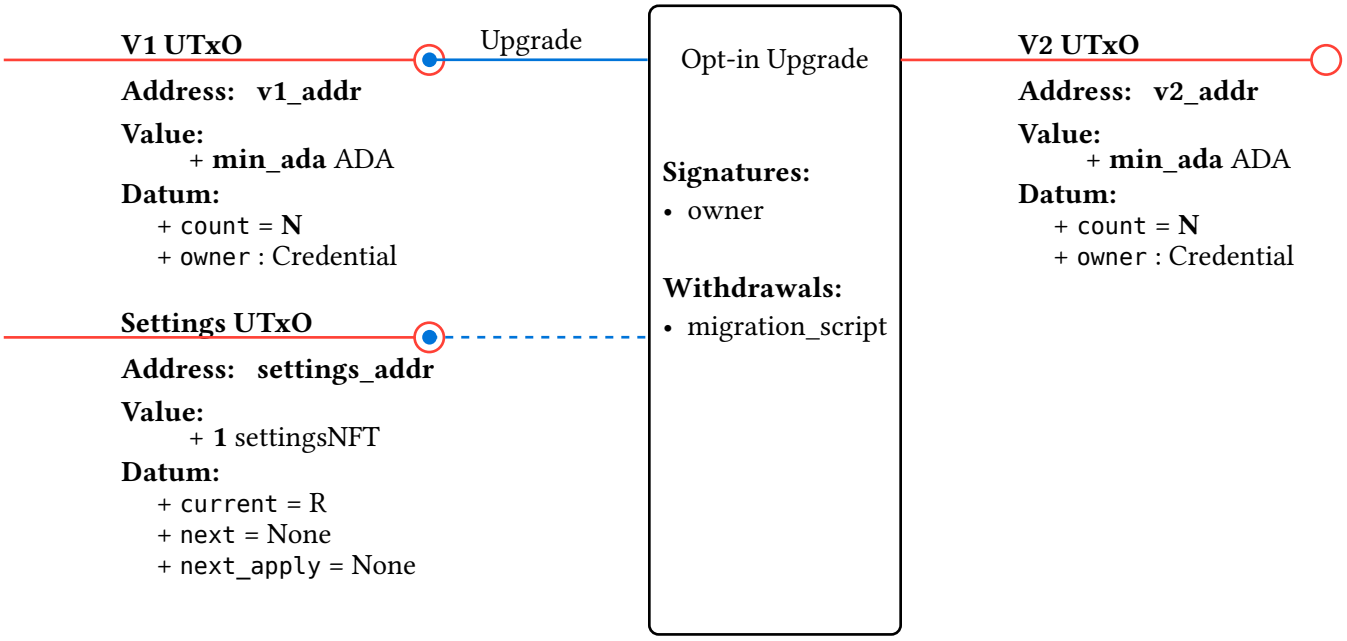


Notes:
- V1's own validator enforces the action natively; the registry is not consulted on this path.

Figure 5: Opt-in native action

7 - Opt-in upgrade (migration)

Moves a user's state to a new immutable version. Consent is enforced natively in V1; the registered migration script validates the transformation.



Notes:

- `validate_upgrade`: `R[v1_addr] = migration_script` must be present **and** `owner` must authorize (consent enforced natively in V1).
- The migration script checks the V1 input was spent with `Upgrade` and that its state reappears at `v2_addr`.
- A governance repoint of `migration_script` changes **how** a migration is performed but can never move funds without `owner`'s signature — this is what makes the mechanism **opt-in**.

Figure 6: Opt-in upgrade transaction